

Spec, Code, Verify

Ganztages-Workshop — Dein eigenes SDD-Framework bauen

Fabian Meyer

Chapter Lead **Vue** @ eventim Tech

Workshop: Agenda & Ziele

Tagesablauf

1 SDD, Context & Live Demo

2 Agent-Harness Werkzeuge

Pause

3 Phase A & B — Bauen

Mittagspause

4 Phase C & D — Erweitern & Testen

Pause

5 Showcase

Ziel des Tages

Ein **funktionierendes SDD-Framework** in eurem eigenen Repo — getestet an einer echten Aufgabe aus eurem Backlog.

Alles was wir heute lernen funktioniert mit Claude Code und Codex gleichermaßen.

Was ist Spec-Driven Development?

Nur ein Buzzword? Jein. In der Essenz sind es 3 Schritte:



Spec

Einen detaillierten Plan mit dem Agenten erstellen, in dem **du** die Entscheidungen triffst.



Code

Den Agenten in einer **neuen Session** implementieren lassen - basierend auf der Spec.



Verify

Den Agenten den Code **gegen mehrere Quality-Gates verifizieren** lassen.

Qualität des Plans = Qualität des Codes.

Das Context Window

Jeder Token in der Session wird bei **jeder Nachricht** verarbeitet.

```
/context
└ Context Usage
  ○○○○○○○○○○○○ Sonnet 4.6
  ○○○○○○○○○○○○ eu.anthropic.claude-sonnet-4-6
  ○○○○○○○○○○○○ 14.5k/200k tokens (7%)
  ○○○○○○○○○○○○ Estimated usage by category
  ○○○○○○○○○○○○ 🛠️ System prompt: 3.3k tokens (1.7%)
  ○○○○○○○○○○○○ 🛠️ System tools: 6.7k tokens (3.3%)
  ○○○○○○○○○○○○ 🧑 Custom agents: 464 tokens (0.2%)
  ○○○○○○○○○○○○ 📄 Memory files: 967 tokens (0.5%)
  ○○○○○○○○○○○○ 🧠 Skills: 3k tokens (1.5%)
  ○○○○○○○○○○○○ 🗨️ Messages: 110 tokens (0.1%)
  ○○○○○○○○○○○○ 🆓 Free space: 152.5k (76.2%)
  ☒☒☒☒☒☒☒☒☒ ☒ Autocompact buffer: 33k tokens (16.5%)
```

`/clear` zwischen Aufgaben

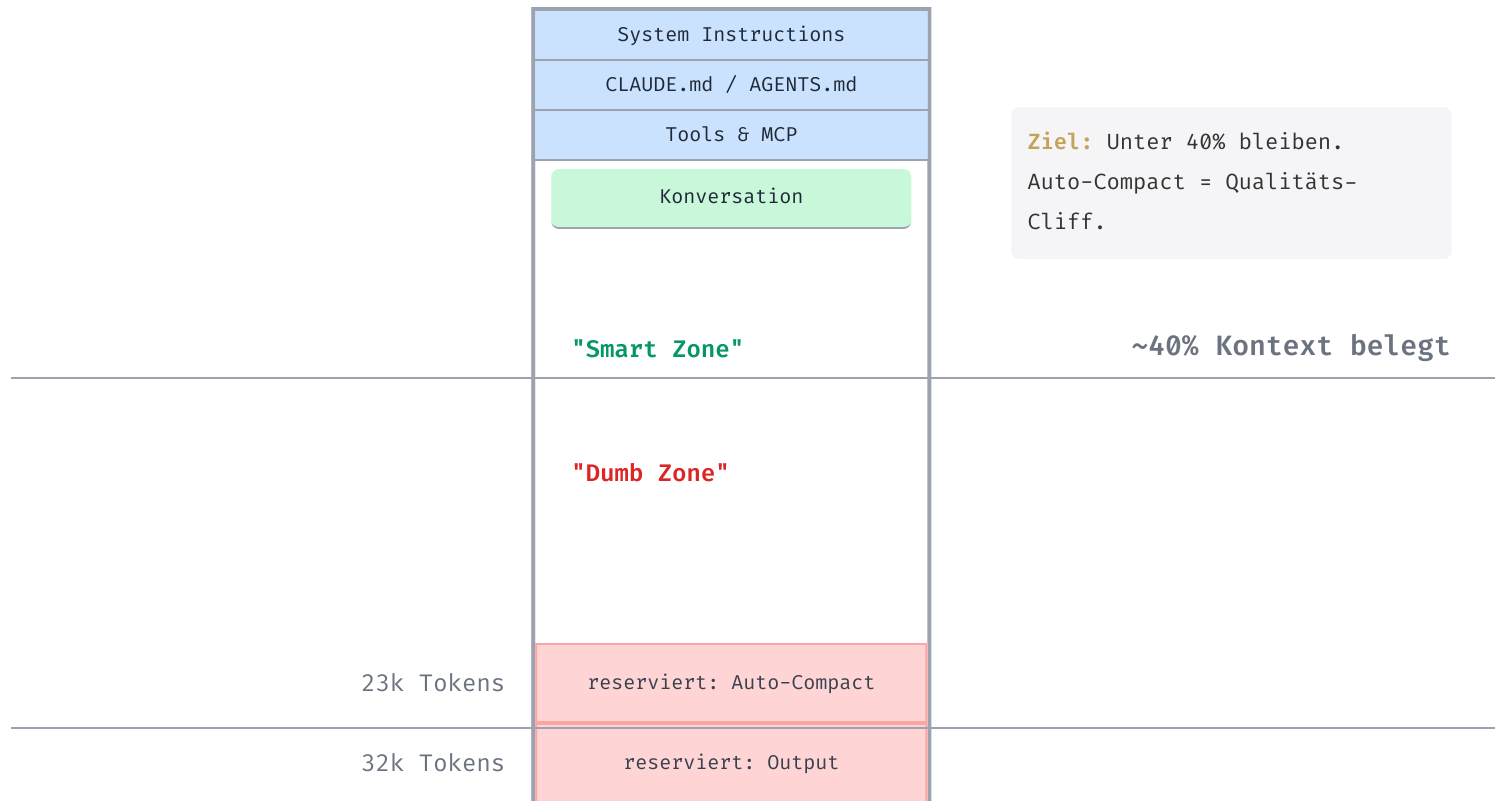
Subagenten isolieren verbosen Output

Specs als Kontext-Transfer



Context Rot: Je voller der Kontext, desto schlechter der Output — LLMs verlieren den Fokus wie Menschen.

Smart Zone vs. Dumb Zone



Spec: Kontext sammeln

 **Was wollen wir tun?** Kontext aus Jira: Ticket, Akzeptanzkriterien, Beschreibung, verlinkte Tickets.

 **Womit arbeiten wir?** Codebase aus Git: verwandte Komponenten, Patterns, Architektur.

 **Brainstorming & Q&A:** Agent erstellt Empfehlungen, geht in Q&A-Session. Gemeinsames Verständnis erarbeiten.

 **Output: Strukturierte Spec.** Recherche, Einschränkungen, Anforderungen, Entscheidungen, Tasks, Verifikationskriterien.

Die Spec ist der Kontext-Transfer. Ohne sie müsste die nächste Session alles nochmal herleiten.

Live Demo

`/spec` auf einem echten Projekt

Brainstorming → Q&A → Strukturierte Spezifikation

Starter-Skills installieren

Holt euch die Workshop-Skills auf euren Rechner — ein Befehl:

```
cd euer-projekt
```

```
npx skills add dino0013/rheinwerk-starter-skills
```

Erkennt euren Agent automatisch und installiert `/spec` und `/dev` ins richtige Verzeichnis:

Claude Code → `.claude/skills/`

Codex → `.agents/skills/`

Ohne Node/npx: `git clone https://github.com/dino0013/rheinwerk-starter-skills /tmp/skills` dann `cp -r /tmp/skills/.claude .` (oder `.agents` für Codex).

Werkzeuge eures Agent-Harness

Jeder Coding Agent bietet dieselben Bausteine — nur die Namen variieren.



Kontext-Datei

Persistente Projekt-
Instruktionen



Skills

Wiederverwendbare
Automationen



Hooks

Deterministische Kontrolle



Subagenten

Delegation & Isolation



Memory

Persistentes Wissen

Kontext-Datei = Was der Agent immer wissen muss. **Skills** = Was der Agent können soll. **Hooks** = Was garantiert passieren muss. **Subagenten** = Parallele/isolierte Arbeit. **Memory** = Was zwischen Sessions erhalten bleibt.

Im nächsten Block schauen wir uns jede Säule im Detail an.

Skill-Anatomie

Ein Skill ist eine `SKILL.md`-Datei mit YAML-Frontmatter und Markdown-Body. Hier der `/spec` Skill aus euren Starter-Skills:

```
---
name: spec
description: >
  Brainstorm and specify a task through
  iterative Q&A. Use when the user wants
  to refine a task, feature idea, or work
  item into a full specification.
argument: <task-description-or-file>
---

# Spec – Task Brainstorming & Specification
```

Frontmatter

`name` — `/spec`
`description` — Auto-Trigger
`argument` — Hint für User

Prinzip

- Frontmatter immer geladen
- Body nur bei Auslösung
- Body = kompletter Workflow

Dateistruktur

`.claude/skills/spec/
SKILL.md`
`.claude/skills/dev/
SKILL.md`

Skill-Auslösung

Skills feuern auf zwei Arten — automatisch und explizit. Der Unterschied: wer entscheidet.

Automatisch

description-triggered

Agent liest `description` + `when_to_use` bei Session-Start.

Wenn dein Request matcht → Skill wird geladen und ausgeführt. Kein `/` nötig.

Ideal für **Referenz-Skills**: Conventions, Patterns, Style Guides.

USER

schreibe eine Angular-Komponente für das Nutzerprofil

SKILL auto-geladen

conventions-angular — Description passt

Explizit

/skill-name

Du tippst `/skill-name` gefolgt von optionalen Argumenten.

Ideal für **Task-Skills**: Deployments, Commits, Multi-Step Workflows.

`disable-model-invocation: true` verhindert Auto-Auslösung.

/fix-issue 123

→ `$ARGUMENTS` = "123"

/deploy "production" v2.1

→ `$0` = "production", `$1` = "v2.1"

\$ARGUMENTS Substitution

`$ARGUMENTS` — alle Args wie getippt

`$0`, `$1`, `$2` — positionelle Args

`$name` — benannte Args (im Frontmatter deklariert)

Die Kontext-Datei

Persistenter Projekt-Kontext, automatisch bei **jeder Session** geladen.

CLAUDE.md

Claude Code

AGENTS.md

Codex

💡 Best Practices

- ✂ Unter 200 Zeilen — keine trivial ableitbaren Infos (package.json, Dateipfade).
- ☰ LLMs können ~300–400 Instruktionen folgen — weniger, schärfere Regeln schlagen lange Listen.
- ✗ Niemals `/init` nutzen — verbrennt Tokens, lenkt ab, veraltet schnell. Selbst schreiben.
- ✎ Erst als Fehler-Dump nutzen — was der Agent falsch macht eintragen, regelmäßig aufräumen.

Die Kontext-Datei ist Teil des nicht-flexiblen Kontexts — jede Zeile kostet bei jeder Nachricht.

Hooks

Deterministische Scripts die bei bestimmten Lifecycle-Events laufen. Anders als die Kontext-Datei (beratend) **garantieren** Hooks die Ausführung.

Konfiguriert in `settings.json` (Claude Code & Codex — gleiche Struktur).



Lifecycle-gesteuert

Trigger auf Events, nicht auf Prompts.



Garantiert

Konfiguration, nicht Instruktion. Der Harness führt sie aus — Agent kann nicht überspringen.



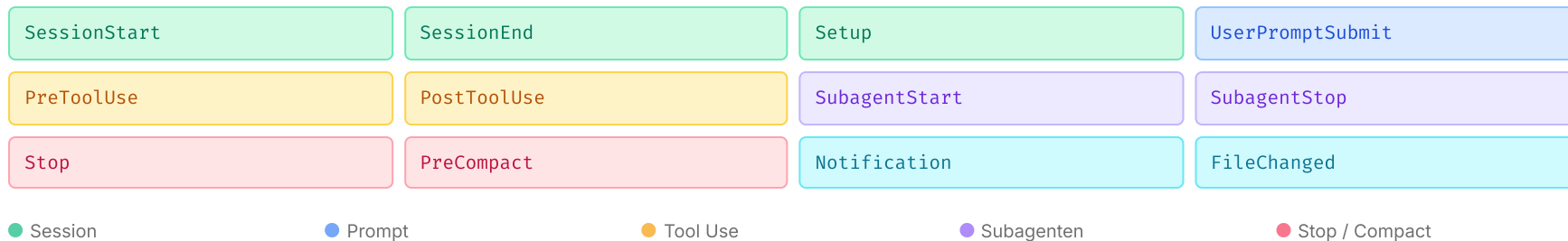
Kann eingreifen

Tools blockieren, Kontext injizieren, Inputs verändern, oder nur beobachten.

💡 Hooks nutzen wenn *"bitte denk daran..."* in der Kontext-Datei nicht reicht.

Hook Events

Events über den gesamten Lifecycle, gruppiert in Kategorien.



Wichtigste für den Alltag: SessionStart (Setup), PreToolUse (validieren/blockieren), PostToolUse (reagieren), Stop (aufräumen).

Hook Lifecycle

Jeder Turn durchläuft diese Events. Die meisten Hooks leben im Agentic Loop.

SessionStart

Projekt-State laden, Caches aufwärmen, Kontext injizieren.

UserPromptSubmit

Prompt validieren oder umschreiben. Kann **blockieren**.

- AGENTIC LOOP -

PreToolUse

[tool executes]

PostToolUse

Pre / PostToolUse

Die mächtigsten Hooks. Gefährliche Tool-Calls blockieren (PreToolUse) oder auf Ergebnisse reagieren — lint, format, scan (PostToolUse).

Stop

Turn-Ende. Tests laufen lassen, Benachrichtigungen, weiterarbeiten erzwingen.

PreCompact / PostCompact

Vor/nach Context-Komprimierung. State sichern, Warnungen, Kontext wiederherstellen.

Subagenten

Spezialisierte Unteragenten mit eigenem Context Window — Delegation und Isolation.

Warum Subagenten?

⚡ ****Kontext-Isolation**** — Verbose Arbeit verschmutzt nicht den Haupt-Kontext.

👤 ****Parallelisierung**** — Mehrere Tasks gleichzeitig bearbeiten.

🎯 ****Spezialisierung**** — Eigener System-Prompt, eigene Tools, eigene Persona.

Faustregel: Wenn eine Aufgabe > 3 Tool-Calls braucht und das Ergebnis in einem Satz zusammenfassbar ist → Subagent.

Agent-Definition

```
# .claude/agents/code-reviewer.md

---
name: code-reviewer
description: >
  Reviews code for bugs, security issues,
  and adherence to project conventions.
  Use for PR reviews and pre-merge checks.
model: sonnet
allowed-tools: Read Bash Grep
```


Memory

Persistentes Wissen das über Sessions hinweg erhalten bleibt — der Agent lernt aus Erfahrung.

Was gehört in Memory?

 ****User-Präferenzen**** — Wie du arbeitest, was du bevorzugst.

 ****Feedback**** — Was der Agent falsch gemacht hat und nicht wiederholen soll.

 ****Projekt-Kontext**** — Nicht-offensichtliche Entscheidungen, laufende Initiativen.

 ****Referenzen**** — Wo externe Infos zu finden sind (Jira, Confluence, Slack).

Was gehört NICHT in Memory?

-  Code-Patterns (→ aus Code ableitbar)
-  Git-History (→ `git log` ist autoritativ)
-  Dateipfade (→ `find / ls` ist aktueller)
-  Debugging-Lösungen (→ Fix ist im Code)
-  Was schon in der Kontext-Datei steht

Prinzip: Memory speichert was *nicht* aus dem aktuellen Projekt-Stand ableitbar ist — Entscheidungen, Präferenzen, Kontext der sonst verloren geht.

Memory ≠ Datenbank. Memory = Erfahrung die den Agenten mit jeder Session besser macht.



Kurze Pause

15 Minuten

Phase A: Anforderungsanalyse

60 Minuten — Euren idealen SDD-Loop entwerfen.

Leitfragen

1. **Täglicher Workflow** — Wie startet ihr ein neues Feature/Ticket?
2. **Wiederholungen** — Welche Schritte wiederholt ihr immer?
3. **Kontext-Verlust** — Wo verliert ihr Kontext zwischen Sessions?
4. **Engpässe** — Was dauert am längsten? Wo passieren Fehler?
5. **Team-Koordination** — Wie koordiniert sich euer Team?
6. **Tools** — Welche Tools brauche ich in welcher Phase?

Output

Entwurf eures idealen SDD-Loops:

```
# Mein SDD-Workflow

## Spec-Phase
—

## Code-Phase
—

## Verify-Phase
—
```

Phase B: Erste Iteration

90 Minuten — Kontext-Datei + ersten Skill bauen und testen.

Schritte

1. Kontext-Datei schreiben für euer Projekt

- Unter 200 Zeilen
- Nur was der Agent nicht selbst findet
- Conventions, Architektur-Entscheidungen, Fehler-Dumps

2. Tools für spec einbinden

- MCP, CLI-Tool

3. Spec-Skill anpassen (oder Äquivalent)

- SKILL.md mit Tool
- Schritte die euer Brainstorming-Loop abbilden (stellt ihr euch projektbezogene Fragen?)

4. Testen auf einer echten Aufgabe aus eurem Backlog

5. Iterieren basierend auf Ergebnis

Tipp: Startet minimal. Ein Skill der 80% abdeckt ist besser als einer der 100% will aber nie fertig wird.



Mittagspause

60 Minuten

Phase C: Erweitern

60 Minuten — Mindestens eine weitere Säule hinzufügen.

Wählt mindestens eins:



/dev Skill

Implementierungs-Automation: Spec lesen → Code schreiben → Tests → Commit.



/verify Skill

Quality Gates: Lint, Tests, Type-Check, Browser-Verifikation gegen die Spec.



Hooks

Pre-Commit Checks, Format-Enforcement, Tool-Validierung.



Subagenten / Memory

Spezialisierte Worker oder persistente Konventionen konfigurieren.

Phase D: End-to-End Test

60 Minuten — Kompletten SDD-Loop auf einer echten Aufgabe durchlaufen.

Ablauf

1. Echte Aufgabe aus eurem Backlog wählen
2. Kompletten Loop durchlaufen: **Spec** → **Code** → **Verify**
3. Probleme notieren, Workflow anpassen
4. Dokumentieren was funktioniert (README oder Notizen)

Checkliste

- ☐ Spec generiert und reviewt
- ☐ Code vom Agenten implementiert
- ☐ Verifikation durchgelaufen
- ☐ Workflow-Probleme notiert
- ☐ Verbesserungen eingebaut
- ☐ Kurze Doku geschrieben

Erwartung: Es wird nicht perfekt laufen. Das ist der Punkt — die Probleme zeigen wo euer Framework noch wachsen muss.



Kurze Pause

10 Minuten

Showcase & Abschluss

Lightning Talks: 3–5 Minuten pro Person/Gruppe.



Euer Loop

Wie sieht euer Workflow aus?



Key Insight

Was hat euch überrascht?



Montag-Morgen

Was setzt ihr als erstes um?

Ressourcen

- **Starter-Skills:** github.com/dinooo13/rheinwerk-starter-skills
- **Claude Code Docs:** docs.anthropic.com/claude-code
- **Codex Docs:** platform.openai.com/docs/codex
- **GitHub:** github.com/dinooo13 — weitere Skills & Beispiele

Danke! Viel Erfolg mit eurem SDD-Framework.